



.Net

Teoría 11

Contenedor de Inyección de dependencias (DI-Container) y Hosting



Crear una solución Hosting



1. Abrir una terminal del sistema operativo
2. Cambiar a la carpeta **proyectosDotnet**
3. Crear una solución **Hosting** con un proyecto classlib llamado **Aplicacion** y un proyecto de consola llamado **ConsolaHost**

```
dotnet new sln -o Hosting
cd Hosting
dotnet new classlib -o Hosting.Aplicacion
dotnet sln add Hosting.Aplicacion
dotnet new console -o Hosting.ConsolaHost
dotnet sln add Hosting.ConsolaHost
dotnet add Hosting.ConsolaHost reference Hosting.Aplicacion
code .
```



Codificar la interfaz ILogger y la clase LoggerConsola en el proyecto Aplicacion



```
//-----ILogger.cs-----
```

```
namespace Hosting.Aplicacion;  
public interface ILogger  
{  
    void Log(string mensaje);  
}
```

```
//-----LoggerConsola.cs-----
```

```
namespace Hosting.Aplicacion;  
public class LoggerConsola : ILogger  
{  
    public void Log(string mensaje)  
    {  
        Console.WriteLine($"{DateTime.Now:HH:mm:ss:fff} {mensaje}");  
    }  
}
```



Codificar la interfaz IServicioX y la clase ServicioX en el proyecto Aplicacion



```
//-----IServicioX.cs-----
```

```
namespace Hosting.Aplicacion;
```

```
public interface IServicioX
```

```
{
```

```
    void Ejecutar();
```

```
}
```

```
//-----ServicioX.cs-----
```

```
namespace Hosting.Aplicacion;
```

```
public class ServicioX (ILogger logger): IServicioX
```

```
{
```

```
    public void Ejecutar()
```

```
    {
```

```
        logger.Log("ServicioX comenzando su ejecución");
```

```
        for (int i = 1; i <= 100_000_000; i++) ; //consumo tiempo simulando ejecución
```

```
        logger.Log("ServicioX ejecución finalizada");
```

```
    }
```

```
}
```

Copiar el código del archivo 11

Teoria-Recursos



Principio de inversión de dependencias Configuración centralizada (Composition Root)

- Para que nuestra aplicación sea fácil de modificar, las clases no deben crear sus propias dependencias (no usar `new` dentro de `ServicioX`).
- Las dependencias deben ser inyectadas desde el exterior (por ejemplo, a través del constructor).
- Idealmente podemos cambiar el comportamiento de la aplicación de esta manera:
 - 1) Crear nuevas clases (dependencias) que implementen las interfaces requeridas.
 - 2) Configurar en un único lugar central qué implementaciones concretas se inyectarán en toda la aplicación.



Principio de inversión de dependencias

Nuestro contenedor manual

- Para concentrar la configuración, vamos a delegar en una única clase la responsabilidad de instanciar y conectar todas las dependencias del sistema.
- Como esta clase pertenece a la configuración de arranque de la aplicación, la crearemos dentro del proyecto `ConsoleHost`.
- A esta clase que actuará como nuestro inyector manual de servicios, la llamaremos `ProveedorServicios`.



Agregar la clase ProveedorServicios en ConsolaHost



```
//-----ProveedorServicios.cs-----
```

```
using Hosting.Aplicacion;
```

```
namespace Hosting.ConsolaHost;
```

```
class ProveedorServicios
```

```
{
```

```
    public ILogger GetLogger()
```

```
        => new LoggerConsola();
```

```
    public IServicioX GetServicioX()
```

```
        => new ServicioX(this.GetLogger());
```

```
}
```

Copiar el código del archivo
11 Teoria-Recursos

Área concentrada del código donde se
configuran todas las dependencias

ProveedorServicios concentra la creación de todas las dependencias

```
class ProveedorServicios
{
    public ILogger GetLogger()
        => new LoggerConsola();
    public IServicioX GetServicioX()
        => new ServicioX(this.GetLogger());
}
```

Para crear un `ServicioX` se necesita un `Logger`. Esto no es problema para `ProveedorServicios` pues también puede autoproporcionárselo





Codificar Program.cs y ejecutar



```
using Hosting.Aplicacion;  
using Hosting.ConsoleHost;
```

```
var proveedor = new ProveedorServicios();  
var servicioX = proveedor.GetServicioX();  
servicioX.Ejecutar();
```

```
var logger = proveedor.GetLogger();  
logger.Log("Fin del programa");
```

Copiar el código del archivo
11 Teoria-Recursos

```
11:29:30:272  ServicioX comenzando su ejecución  
11:29:30:621  ServicioX ejecución finalizada  
11:29:30:621  Fin del programa
```



Contenedor de Inyección de Dependencias

- En lugar de nuestra clase manual `ProveedorServicios` utilizada en el ejemplo anterior, usaremos el `contenedor de inyección de dependencias` oficial de .NET.
- Un contenedor de inyección de dependencias (`DI Container`) facilita configurar y obtener las dependencias que se usarán en la aplicación.
- También permite especificar el `ciclo de vida de una dependencia`, es decir, si debe crearse un nuevo objeto cada vez o si debe reutilizarse.

Ciclos de Vida en el Contenedor

- **Transient (Transitorio):** El contenedor creará una nueva instancia del objeto cada vez que una clase lo solicite.
- **Singleton:** Se instanciará un único objeto la primera vez que se solicite. A partir de ahí, la aplicación trabajará siempre con la misma instancia en cualquier parte del código. (Singleton es también un reconocido patrón de diseño).





Contenedor de Inyección de Dependencias

- .NET provee su propio contenedor por medio de las clases `ServiceCollection` y `ServiceProvider`.
- Para utilizar estas clases en nuestra aplicación de consola, es necesario instalar un `paquete NuGet` adicional.
- `NuGet` es el administrador de paquetes gratuito y de código abierto diseñado para el ecosistema .NET.



Instalación del paquete en ConsolaHost



- En la terminal del sistema operativo, debemos posicionarnos **en la carpeta de nuestro proyecto de consola** (donde se encuentra el archivo `Hosting.ConsolaHost.csproj`) y tipear el siguiente comando:

```
dotnet add package Microsoft.Extensions.Hosting
```

Este es el nombre del paquete
que se requiere instalar

(Nota: Este paquete incluye internamente todo lo necesario para inyección de dependencias, configuración y logging).

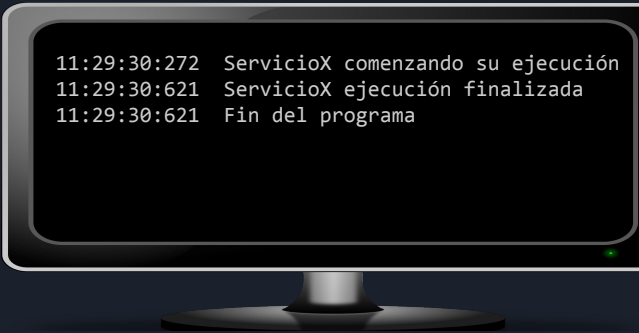


Modificar Program.cs (ConsolaHost) y ejecutar



```
using Hosting.Aplicacion;  
using Microsoft.Extensions.DependencyInjection;  
  
var servicios = new ServiceCollection();  
  
// Registramos los servicios  
servicios.AddTransient<ILogger, LoggerConsola>();  
servicios.AddTransient<IServicioX, ServicioX>();  
  
// Construimos el proveedor  
using var proveedor = servicios.BuildServiceProvider();  
  
// Solicitamos los servicios al contenedor  
var servicioX = proveedor.GetService<IServicioX>();  
servicioX?.Ejecutar();  
  
var logger = proveedor.GetService<ILogger>();  
logger?.Log("Fin del programa con DI Container oficial");
```

Agregar esta
directiva using



```
11:29:30:272 ServicioX comenzando su ejecución  
11:29:30:621 ServicioX ejecución finalizada  
11:29:30:621 Fin del programa
```

La clase
ProveedorServicios
ya no es necesaria

Se registran los servicios y se
construye el proveedor

Copiar el código del archivo
11 Teoria-Recursos



Nota técnica:

La variable `proveedor` es del tipo `ServiceProvider`, el cual implementa la interfaz `IDisposable`. Al utilizar la declaración `using`, nos aseguramos de que se ejecute su método `Dispose()` al finalizar el programa.

Esto es fundamental, ya que al destruirse, el contenedor se encarga automáticamente de liberar (hacer `Dispose`) a todos los servicios que él haya instanciado y que requieran liberación de recursos, previniendo así fugas de memoria.

```
// Construimos el proveedor
using var proveedor = servicios.BuildServiceProvider();

// Solicitamos los servicios al contenedor
var servicioX = proveedor.GetService<IServicioX>();
servicioX?.Ejecutar();

var logger = proveedor.GetService<ILogger>();
logger?.Log("Fin del programa con DI Container oficial");
```



Descripción del código presentado

```
servicios.AddTransient<IServicioX, ServicioX>();  
servicios.AddTransient<ILogger, LoggerConsola>();
```

Se registran las interfaces `IServicioX` e `ILogger` en la colección de servicios. Al usar `AddTransient`, indicamos que cuando alguna clase requiera un `IServicioX`, el contenedor debe hacer un `new` y proveer una nueva instancia de la clase `ServicioX`. Lo mismo aplica para `LoggerConsola`.

Descripción del código presentado

```
using var proveedor = servicios.BuildServiceProvider();  
  
var servicio = proveedor.GetService<IServicioX>();
```

Se obtiene el **proveedor de servicios** (el motor que fabrica los objetos) a partir de la colección donde registramos las "recetas" es decir *"para esta interfaz devolver esta clase"*

Se instancia y devuelve un objeto de la clase **ServicioX**.

- No debemos preocuparnos por las **dependencias** que requiere **ServicioX**, serán provistas por el contenedor (**Importante: esas dependencias también deben estar registrada en el contenedor**).
- Si **ServicioX** es disponible, tampoco nos preocuparemos por hacerle el **Dispose()**. Quien crea el objeto es responsable de hacerlo. En este caso será el **Proveedor de Servicios** lo hará.

Reglas de Oro del Contenedor de DI

Para que el **contenedor** pueda proveer los servicios requeridos se necesita:

1. **Haber registrado** previamente **el servicio y todas sus dependencias** (y las dependencias de sus dependencias) en el contenedor (**ServiceCollection**).
2. Utilizar en todos los casos la **inyección por medio del constructor** en nuestras clases.



Nota sobre el patrón Builder

```
var servicios = new ServiceCollection();  
... // aquí se configura el objeto que se construirá  
var proveedor = servicios.BuildServiceProvider();
```

- Aquí estamos utilizando el patrón **Builder** (Constructor), que permite construir un objeto complejo (en este caso un **ServiceProvider**) paso a paso, separando el proceso de configuración del resultado final.
- **ServiceCollection** actúa como el builder.

Nota sobre el patrón Builder

¿Cual puede ser la ventaja de usar el patrón Builder aquí?

¿Por qué no instanciar directamente el `ServiceProvider` con un `new` y luego configurarle los servicios por medio de métodos o propiedades públicas?



Ventajas de usar el patrón Builder

- **Separación de Responsabilidades:** Separa claramente la fase de arranque y configuración (**Setup**) del código de uso de la aplicación (**Runtime**).
- **Inmutabilidad:** Una vez que el objeto final es construido (**BuildServiceProvider**), la colección de dependencias se vuelve inmutable. No se pueden agregar servicios "al vuelo" mientras la aplicación se está ejecutando, lo que previene errores impredecibles.
- **Modularidad y Extensibilidad (Configuración Distribuida):** Permite delegar la configuración a diferentes componentes. El **builder** actúa como un "colector" central que viaja a través de distintas partes de la aplicación para que cada una haga su aporte a la configuración final del objeto antes de su creación.





Ejercicio práctico: Se requiere numerar las líneas de log



- Agregar un nuevo servicio `LoggerNum` en el proyecto `Aplicacion` que implemente la interfaz `ILogger` y que enumere secuencialmente las líneas que va imprimiendo en la consola.



Ejercicio práctico: Se requiere numerar las líneas de log



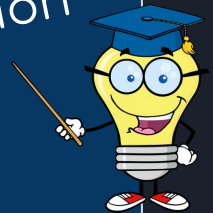
- Agregar un nuevo servicio **LoggerNum** en el proyecto **Aplicacion** que implemente la interfaz **ILogger** y que enumere secuencialmente las líneas que va imprimiendo en la consola.

```
namespace Hosting.Aplicacion;

public class LoggerNum : ILogger
{
    private int _n;

    public void Log(string mensaje)
    {
        Console.WriteLine($"{++_n}: {DateTime.Now:HH:mm:ss:fff} {mensaje}");
    }
}
```

Posible
solución





Ejercicio práctico: Se requiere numerar las líneas de log



- Agregar un nuevo servicio `LoggerNum` en el proyecto `Aplicacion` que implemente la interfaz `ILogger` y que enumere secuencialmente las líneas que va imprimiendo en la consola.
- En `Program.cs` del proyecto `ConsolaHost` realizar este único cambio: reemplazar `LoggerConsola` por `LoggerNum` en la instrucción de registro. Luego compilar y ejecutar para verificar el resultado

```
. . .  
// Registramos los servicios  
servicios.AddTransient<ILogger, LoggerNum>();  
servicios.AddTransient<IServicioX, ServicioX>();  
. . .
```



Ejercicio práctico: El resultado inesperado



¡Atención! Observar que la numeración de la tercera línea del log es **incorrecta**. Vuelve a empezar en **1**. ¿Por qué?

Porque registramos el servicio como **Transient**. El contenedor creó un objeto **LoggerNum** para dárselo a **ServicioX**, y luego creó un **nuevo** objeto **LoggerNum** (con su propia variable **_n** en cero) para la línea final del **Program.cs**.

Mal



```
1: 12:40:43:981  ServicioX comenzando su ejecución
2: 12:40:44:284  ServicioX ejecución finalizada
1: 12:40:44:285  Fin del programa
```



Ejercicio práctico: La solución



- Agregar un nuevo servicio **LoggerNum** en el proyecto **Aplicacion** que implemente la interfaz **ILogger** y que enumere secuencialmente las líneas que va imprimiendo en la consola.
- En **Program.cs** realizar este único cambio: reemplazar **LoggerConsola** por **LoggerNum** en la instrucción de registro. Luego compilar y ejecutar para verificar el resultado
- Registrar el servicio de Logger como Singleton en lugar de Transient, usando la instrucción:

```
servicios.AddSingleton<ILogger, LoggerNum>();
```

(Ejecutar y verificar que ahora la tercera línea del log dirá 3: correctamente).

Principio de inversión de dependencias - Configuración de las dependencias - DI container

-----Program.cs-----

```
using Hosting.Aplicacion;
using Microsoft.Extensions.DependencyInjection;

var servicios = new ServiceCollection();

// Registramos los servicios
servicios.AddSingleton<ILogger, LoggerNum>();
servicios.AddTransient<IServicioX, ServicioX>();

// Construimos el proveedor
var proveedor = servicios.BuildServiceProvider();

// Solicitamos los servicios al contenedor
var servicioX = proveedor.GetService<IServicioX>();
servicioX?.Ejecutar();

var logger = proveedor.GetService<ILogger>();
logger?.Log("Fin del programa con DI Container oficial");
```

Se necesita siempre acceder a la misma instancia del servicio de log, por eso lo registramos como Singleton

Bien →

```
1: 12:51:41:857  ServicioX comenzando su ejecución
2: 12:51:42:164  ServicioX ejecución finalizada
3: 12:51:42:165  Fin del programa
```

Resumiendo

- `servicios.AddTransient<ILogger, LoggerNum>();`
Registra el servicio como **transitorio**. El proveedor devolverá un **nuevo objeto** cada vez que se lo requiera. Útil para servicios ligeros y sin estado.
- `servicios.AddSingleton<ILogger, LoggerNum>();`
Registra el servicio como **singleton**. El proveedor devolverá **siempre el mismo objeto** cada vez que se lo requiera durante toda la vida de la aplicación. Útil para compartir estado o recursos costosos de crear.

El valor de la Inyección de Dependencias

Hemos cambiado el comportamiento del programa agregando nuevo código y modificando sólo el área donde se registran los servicios.

¡El resto de la aplicación (**ServicioX**) ni se enteró del cambio!

ii OCP (Open/Close Principle) !!



El valor de la Inyección de Dependencias

¿Siempre debemos crear una nueva clase para modificar un comportamiento? La respuesta en el desarrollo real es: **NO**.

- Si sabemos con certeza que ninguna otra parte del sistema volverá a utilizar `LoggerConsola`, dejar esa clase abandonada en el código es una mala práctica.
- Aquí entra en juego el principio **YAGNI** (*You Aren't Gonna Need It* - No lo vas a necesitar). No debemos acumular código muerto.
- **En la práctica:** Si el código nos pertenece y el comportamiento anterior es obsoleto para toda la aplicación, lo más limpio y fácil de mantener es **refactorizar** directamente la clase original en lugar de crear una nueva.
- El principio **OCP** brilla realmente cuando desarrollamos librerías consumidas por terceros, o cuando necesitamos que *convivan* múltiples implementaciones al mismo tiempo.



Registrando clases concretas

NOTA

También es posible registrar clases directamente, sin especificar la interfaz que implementan. Supongamos que queremos registrar un caso de uso específico:

```
var servicios = new ServiceCollection();
servicios.AddTransient<CrearUsuarioUseCase>();

ServiceProvider proveedor = servicios.BuildServiceProvider();
var crearUsuario = proveedor.GetService<CrearUsuarioUseCase>();
// aquí podemos utilizar la instancia provista por proveedor
```



Hosting

El patrón de hospedaje

El patrón de hospedaje (Generic Host en .NET)

Hasta ahora instanciamos nuestro contenedor de dependencias manualmente utilizando `new ServiceCollection()`. Esto funciona perfectamente para aprender, pero las aplicaciones reales en producción requieren gestionar mucha más infraestructura.

El patrón de hospedaje (**Generic Host** en .NET) provee un objeto central que encapsula y administra todos los recursos de la aplicación de forma unificada.

Al utilizar un **Host**, delegamos en el framework la configuración inicial de cuatro pilares fundamentales:

1. **Inyección de Dependencias:** El contenedor (**DI Container**) que ya conocemos.
2. **Configuración:** Lectura automática de archivos externos (ej. `appsettings.json`) y **variables de entorno** del **Sistema Operativo**.
3. **Logging:** Un sistema centralizado para registrar eventos y errores.
4. **Ciclo de vida:** Control ordenado del arranque y apagado de la aplicación.

Separación de Responsabilidades

- Siguiendo principios de diseño limpio, la clase `Program.cs` debe considerarse un `componente de bajo nivel`. Su `única función` debe ser `configurar el arranque del sistema` (el *Composition Root*) y luego ceder el control.
- La `lógica` real de nuestra aplicación **NO debe conocer** cómo se configuró el contenedor. Para lograr este aislamiento, vamos a crear una clase intermediaria llamada `AppRunner` dentro del proyecto `ConsoleHost`.
- *`AppRunner` recibirá por inyección de dependencias aquellos objetos que necesite. No utilizará la palabra clave `new`. Pedirá todas sus dependencias directamente a través de su constructor primario.*



Codificar la clase AppRunner dentro del proyecto ConsoleHost



```
using Hosting.Aplicacion;

namespace Hosting.ConsoleHost;

public class AppRunner(IServicioX servicioX, ILogger logger)
{
    public void Ejecutar()
    {
        logger.Log(">>> Iniciando la aplicación desde AppRunner <<<");

        // Ejecutamos la lógica de negocio central
        servicioX.Ejecutar();

        logger.Log(">>> Aplicación finalizada en AppRunner <<<");
    }
}
```

Refactorizando Program.cs (El Builder)

Utilizando el paquete **Microsoft.Extensions.Hosting** que ya instalamos, cambiaremos la forma en que arrancamos nuestra aplicación de consola basándonos en dos conceptos clave:

- **El Builder (HostApplicationBuilder):** Es el objeto responsable de preparar y configurar todo el entorno. Es nuestra herramienta de configuración para registrar los servicios y cargar los archivos de propiedades antes de arrancar.
- **El Host (IHost):** Una vez finalizada la fase de configuración, le pedimos al **builder** que construya el **host**. Este es el objeto definitivo que encapsula todo lo necesario y tiene la aplicación lista para ser ejecutada.



Refactorizar Program.cs del proyecto ConsoleHost y ejecutar



```
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.DependencyInjection;
using Hosting.Aplicacion;
using Hosting.ConsoleHost;

// 1. Creamos el constructor de la aplicación
var builder = Host.CreateApplicationBuilder(args);
```

```
// 2. Registramos los servicios en el contenedor (builder.Services es un IServiceCollection)
builder.Services.AddSingleton<ILogger, LoggerNum>();
builder.Services.AddTransient<IServicioX, ServicioX>();

// 3. Registramos el AppRunner para que sea resuelto por el contenedor
builder.Services.AddTransient<AppRunner>();

// 4. Construimos el Host (aquí se "cierra" la configuración y se valida el árbol de DI)
using IHost app = builder.Build();

// 5. Solicitamos la instancia de AppRunner al proveedor de servicios del Host
var runner = app.Services.GetRequiredService<AppRunner>();

// 6. Cedemos el control a nuestra clase principal para ejecutar la aplicación
runner.Ejecutar();
```

Nota técnica

`builder.Services` es un `IServiceCollection`

`app.Services` es un `IServiceProvider`



Refactorizar Program.cs del proyecto ConsoleHost y ejecutar



```
usi
usi
usi
usi
1: 16:05:06:305 >>> Iniciando la aplicación desde AppRunner <<<
2: 16:05:06:329 ServicioX comenzando su ejecución
//
3: 16:05:06:408 ServicioX ejecución finalizada
var
4: 16:05:06:408 >>> Aplicación finalizada en AppRunner <<<
```

```
//
bui
bui
```

```
//
bui
```

```
//
usi
```

```
//
var runner = app.Services.GetRequiredService<AppRunner>();
```

```
// 6. Cedemos el control a nuestra clase para ejecutar la aplicación
runner.Ejecutar();
```


Configuración externa con appsettings.json

- **El problema del "hardcodeo":** No es buena práctica dejar valores fijos en el código cuando:
 - Varían según el entorno de ejecución (Desarrollo, Pruebas, Producción).
 - Necesitan ajustarse en caliente sin tener que modificar código ni recompilar.
- **La solución arquitectónica:** Externalizar estos valores en archivos de configuración. Crearemos `appsettings.json` en la raíz del proyecto para centralizarlos.
- **Caso de uso práctico:** Lo usaremos para indicarle a nuestro `log` en qué número de línea debe comenzar a enumerar.
- 💡 **Ventaja clave:** Modificando este archivo y reiniciando la aplicación, el comportamiento cambiará al instante. **Cero cambios en C#, cero recompilaciones.**



Codificar appsettings.json en la raíz del proyecto ConsoleHost



```
{  
  "LoggerConfig": {  
    "LineaInicial": 100  
  }  
}
```

Precaución

El nombre del archivo debe ser exactamente **appsettings.json** (todo en minúscula) y debe estar en la raíz del proyecto ConsoleHost.

Copiar el código del archivo

11 Teoria-Recursos

Configuración externa con appsettings.json

Necesitamos configurar `LoggerNum` a partir de `appsettings.json` preservando la independencia de la capa de Aplicación

- `LoggerNum` es una clase de la capa de `Aplicación`.
- Esta capa no debe depender de la infraestructura de configuración que instalamos en el `proyecto de consola` (mediante `Microsoft.Extensions.Hosting`)

Solución: Crear `LoggerNumOptions` dentro de la capa de `Aplicación`.

- Única responsabilidad: contener los datos necesarios para configurar `LoggerNum`.
- Una instancia de `LoggerNumOptions` (gestionada por el `contenedor de DI`) se inyectará en `LoggerNum`



En la capa de Aplicación crear la clase LoggerNumOptions y modificar LoggerNum



En LoggerNumOptions.cs

```
namespace Hosting.Aplicacion;

public class LoggerNumOptions
{
    public int LineaInicial { get; set; }
}
```

En LoggerNum.cs

```
namespace Hosting.Aplicacion;

public class LoggerNum(LoggerNumOptions options):ILogger
{
    private int _n = options.LineaInicial;
    public void Log(string mensaje)
    {
        Console.WriteLine($"{++_n}: {DateTime.Now:HH:mm:ss:fff} {mensaje}");
    }
}
```

Copiar el código del archivo
11 Teoria-Recursos



En el proyecto de consola modificar program.cs



```
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.DependencyInjection;
using Hosting.Aplicacion;
using Hosting.ConsolaHost;
using Microsoft.Extensions.Configuration;

// 1. Creamos el constructor de la aplicación
var builder = Host.CreateApplicationBuilder(args);
```

```
// 2. Registramos los servicios en el contenedor (builder.Services es un IServiceCollection)
builder.Services.AddSingleton<ILogger, LoggerNum>();
builder.Services.AddTransient<IServicioX, ServicioX>();
```

```
// 2.a) Leemos la sección del JSON y la transformamos en un objeto LoggerNumOptions
var loggerOptions = builder.Configuration
    .GetSection("LoggerConfig")
    .Get<LoggerNumOptions>() ?? new LoggerNumOptions();
```

```
// 2.b) Registramos la instancia para que el contenedor la pueda inyectar
builder.Services.AddSingleton(loggerOptions);
```

```
// 3. Registramos el AppRunner para que sea resuelto por el contenedor
builder.Services.AddTransient<AppRunner>();
```

```
// . . . El resto sin modificaciones
```

Archivo appsettings.json

```
{
  "LoggerConfig": {
    "LineaInicial": 100
  }
}
```

Copiar el código del archivo
11 Teoria-Recursos



En el proyecto de consola modificar program.cs



Nota Técnica: Registrando Instancias Directas

- Hasta ahora le dábamos al contenedor **Interfaces** o **Clases** para que él mismo fabrique los objetos cuando sea necesario.
- Al hacer `AddSingleton(loggerOptions)`, estamos haciendo algo distinto: le estamos entregando un objeto que **ya fue creado y existe en memoria**.
- El contenedor no fabricará nada nuevo; simplemente entregará esta instancia exacta a cualquier clase que solicite un `LoggerOptions`.

```
// 2.b) Registramos la instancia para que el contenedor la pueda inyectar  
builder.Services.AddSingleton(loggerOptions);
```

```
// 3. Registramos el AppRunner para que sea resuelto por el contenedor  
builder.Services.AddTransient<AppRunner>();
```

```
// . . . El resto sin modificaciones
```

Copiar el código del archivo
11 Teoria-Recursos

Nota Técnica: Ejecución y el Directorio de Trabajo

Para que el *Host* encuentre exitosamente nuestro `appsettings.json`, el **Directorio de Trabajo** (*Working Directory*) al momento de arrancar la aplicación debe ser la carpeta raíz del proyecto de consola. Tenemos distintas alternativas para ejecutar el código:

1. **Desde la Terminal (dotnet run):** Posicionados en la carpeta del proyecto de consola, tipeamos `dotnet run` para compilar y ejecutar la aplicación utilizando esa misma carpeta como directorio de trabajo. El archivo `JSON` se encontrará sin problemas.
2. **Depurador de VS Code (F5):** Si generamos los recursos de depuración (la carpeta `.vscode` con el archivo `launch.json`), este archivo ya viene preparado para establecer la raíz del proyecto como el directorio de trabajo correcto.
3. **Botón "Ejecutar" (Arriba a la derecha en VS Code):** ¡Atención con esta opción! Suele ejecutar el programa situándose directamente en la carpeta de salida de la compilación (dentro de `bin/Debug/...`). Como nosotros no le indicamos al compilador que copie el `appsettings.json` a esa carpeta, el *Host* no lo encontrará. Para que funcione por esta vía, deberíamos copiar el archivo allí.



El Estándar de la Industria: Automatizar la copia de appsettings.json y ejecutar con el botón ▶



Le indicaremos al compilador de .NET que lo copie el archivo `appsettings.json` a la carpeta de salida (`bin/Debug/...`) cada vez que se construya el proyecto.

Abrir el archivo `Hosting.ConsoleHost.csproj` y agregar el siguiente bloque `<ItemGroup>`:

```
<ItemGroup>
  <None Update="appsettings.json">
    <CopyToOutputDirectory>PreserveNewest</CopyToOutputDirectory>
  </None>
</ItemGroup>
```

 **Nota:** Al guardar este cambio y volver a compilar, el archivo `appsettings.json` se copiará a la carpeta donde se encuentra el ejecutable ¡Problema resuelto de forma permanente!



El Estándar de la Industria: Automatizar la copia de appsettings.json y ejecutar con el botón ▶



```
101: 16:58:18:367 >>> Iniciando la aplicación desde AppRunner <<<
102: 16:58:18:391 ServicioX comenzando su ejecución
103: 16:58:18:467 ServicioX ejecución finalizada
104: 16:58:18:467 >>> Aplicación finalizada en AppRunner <<<
```

appsettings.json se copiará a la carpeta de ejecución donde se encuentra el ejecutable ¡Problema resuelto de forma permanente!

Copiar el código del archivo

11 Teoría-Recursos

El tercer ciclo de vida: Scoped (Ámbito)

Además de `Transient` y `Singleton`, los servicios también se pueden registrar dentro de un ámbito temporal o "`Scope`" usando `AddScoped<>`.

Este ciclo de vida es el pilar de las aplicaciones Web. En `ASP.NET Core`, el framework crea automáticamente un `Scope` nuevo en el instante en que recibe una petición `HTTP` desde un cliente.

- **Durante la petición:** Si varias clases solicitan una misma dependencia a lo largo de esa solicitud, el contenedor les entregará exactamente la misma instancia.
- **Al finalizar la petición:** Una vez que el servidor termina de procesar la solicitud y envía la respuesta al cliente, **el `Scope` se destruye automáticamente**. Todas las instancias creadas dentro de ese ámbito son liberadas de la memoria.
- **Aislamiento:** Si entra una nueva petición `HTTP` (del mismo o de otro usuario), se genera un `Scope` totalmente nuevo y limpio.

Web API

Introducción a ASP.NET Core



¿Qué es ASP.NET Core?

- Es el **framework** oficial de **Microsoft**, gratuito, de código abierto y multiplataforma, diseñado para construir **aplicaciones y servicios web** modernos.
- Una **Web API** es un tipo de aplicación que se construye sobre **ASP.NET Core**.
- Al crear un proyecto web, pasamos a heredar automáticamente toda la infraestructura nativa del **framework** (**servidor web**, **manejo de rutas** y, fundamentalmente, el **contenedor de inyección de dependencias** que acabamos de usar en la consola).



Integrando la Web API a nuestra solución



Vamos a agregar un nuevo proyecto a nuestra solución que actuará como nuestro "ApiHost".

En la terminal, **ubicados en la carpeta raíz de la solución Hosting** ejecutar estos comandos:

```
# Creamos la Minimal API
dotnet new web -o Hosting.ApiHost

# La agregamos a la solución
dotnet sln add Hosting.ApiHost

# Le agregamos la referencia a nuestra capa de Aplicación
dotnet add Hosting.ApiHost reference Hosting.Aplicacion
```

Explorando nuestra primera Minimal API

El comando `dotnet new web` nos generó una plantilla básica en `Program.cs`. Analicemos qué hace cada línea:

```
// 1. Crea el "Constructor" (Builder) del Host web.  
// Aquí adentro ya viene lista la colección de servicios (builder.Services)  
// para que registremos nuestras dependencias.  
var builder = WebApplication.CreateBuilder(args);  
  
// 2. Construye la aplicación web y "cierra" la etapa de configuración  
var app = builder.Build();  
  
// 3. Define un "Endpoint" (Ruta). Cuando el navegador haga una petición  
// a la URL raíz ("/"), la API responderá con el texto "Hello World!"  
app.MapGet("/", () => "Hello World!");  
  
// 4. Arranca el servidor web interno (Kestrel) y se queda escuchando  
// peticiones  
app.Run();
```




Ejecutar ApiHost en VS Code



1. Abrir el archivo `Program.cs` del proyecto `ApiHost` para que quede en la pestaña activa (en primer plano).
2. Hacer clic en el botón de **Ejecutar** (el triángulo en la esquina superior derecha) provisto por la extensión `C# Dev Kit`.

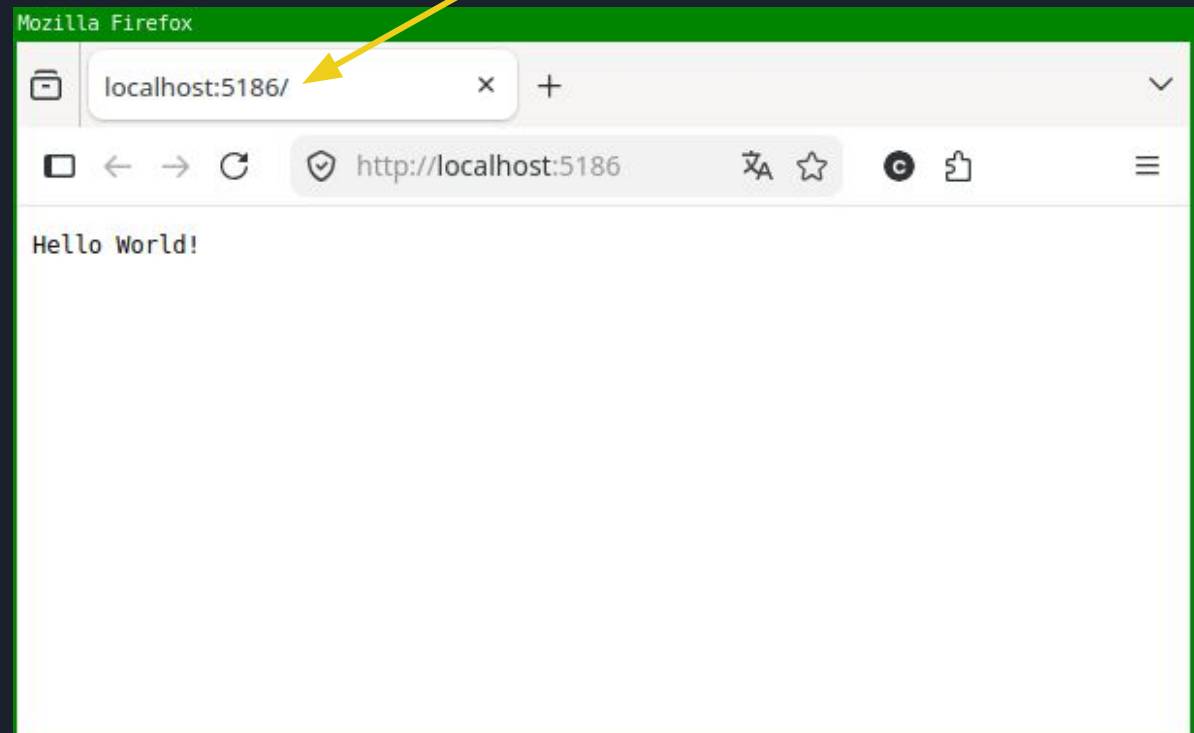
Nota importante sobre los Assets de Depuración (.vscode):

Si al principio del desarrollo generamos esos "assets" de depuración con el comando `.NET: Generate Assets for Build and Debug` cuando solo existía `ConsoleHost`. Al tener ahora dos ejecutables, tenemos dos opciones:

1. **Opción rápida:** Olvidarnos de `F5` por un momento y usar siempre el botón triangular manteniendo el `Program.cs` de `ApiHost` en primer plano.
2. **Opción definitiva:** Borrar por completo la carpeta `.vscode`, presionar `Ctrl+Shift+P` para abrir la paleta de comandos, volver a ejecutar `.NET: Generate Assets for Build and Debug` y seleccionar el proyecto web para que `F5` quede configurado con la API.

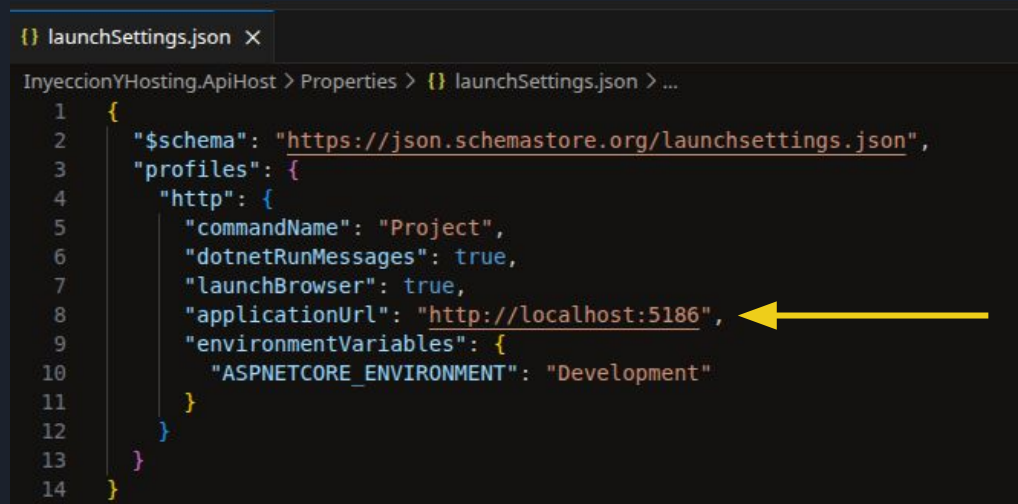
En la TERMINAL o DEBUG CONSOLE de VS Code

```
info: Microsoft.Hosting.Lifetime[14]
  Now listening on: http://localhost:5186
info: Microsoft.Hosting.Lifetime[0]
  Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
  Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
  Content root path: ../Hosting/Hosting.ApiHost
```



¿De dónde sale el puerto de la URL?

- Durante el comando `dotnet new web` se genera un puerto aleatorio (superior a 5000) donde va a escuchar la aplicación.
- Esta configuración se almacena en el archivo del proyecto: **ApiHost / Properties / launchSettings.json**. Si se omite la configuración, el puerto predeterminado es el 5000.
- El puerto se define en la propiedad "`applicationUrl`" bajo el perfil "`http`".

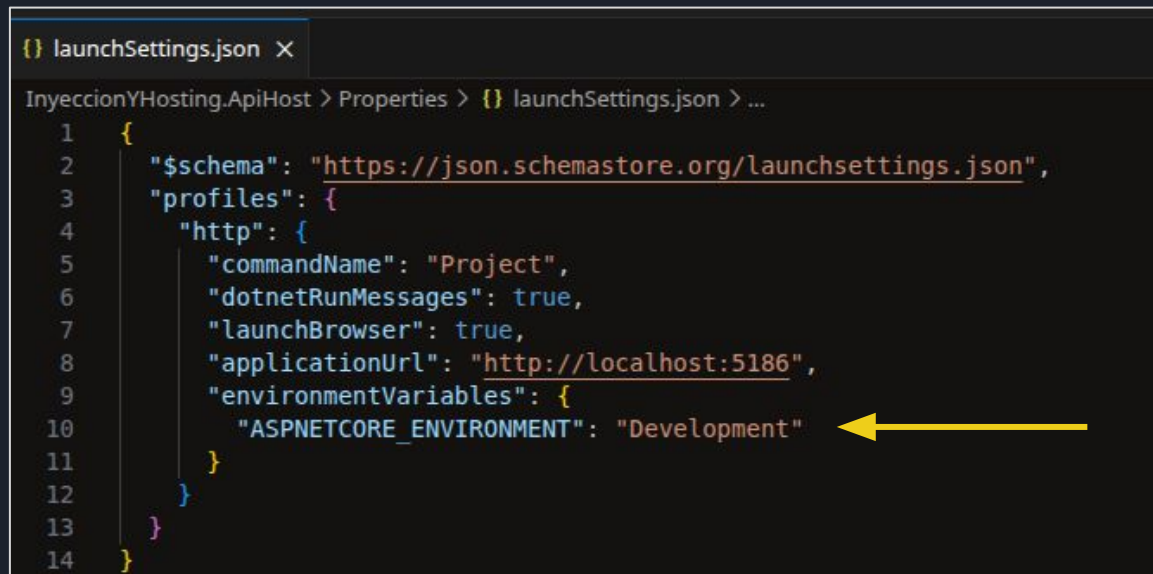


```
{
  "$schema": "https://json.schemastore.org/launchsettings.json",
  "profiles": {
    "http": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "applicationUrl": "http://localhost:5186",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}
```

Entorno de Ejecución

En el archivo `launchSettings.json` se define una variable de entorno crucial para el ecosistema de ASP.NET Core:

`"ASPNETCORE_ENVIRONMENT": "Development"`



```
{ launchSettings.json X
InyeccionYHosting.ApiHost > Properties > {} launchSettings.json > ...
1  {
2    "$schema": "https://json.schemastore.org/launchsettings.json",
3    "profiles": {
4      "http": {
5        "commandName": "Project",
6        "dotnetRunMessages": true,
7        "launchBrowser": true,
8        "applicationUrl": "http://localhost:5186",
9        "environmentVariables": {
10         "ASPNETCORE_ENVIRONMENT": "Development"
11       }
12     }
13   }
14 }
```

Nuestro código puede leer esta variable en tiempo de ejecución para activar o desactivar componentes según el entorno en el que se encuentre.

Entorno de Ejecución

En el desarrollo web profesional, una aplicación transita por diferentes etapas o **entornos**:

1. **Development (Desarrollo):** La máquina del programador. Se permiten mensajes de error detallados y herramientas de prueba.
2. **Staging (Pre-producción o Pruebas):** Un servidor idéntico al real donde el equipo de calidad (**QA**) testea la aplicación de forma integral.
3. **Production (Producción):** El entorno real donde interactúan los usuarios finales. Los errores técnicos se ocultan por motivos de seguridad y se prioriza el rendimiento.



El Sistema de Configuración Unificado

Existe un **sistema jerárquico** que busca valores en múltiples fuentes y los unifica automáticamente en `builder.Configuration`.

Cuando la misma clave existe en más de un lugar, **la última fuente leída tiene prioridad y sobrescribe (override) a las anteriores**. El orden estándar de menor a mayor prioridad es:

1. **`appsettings.json` (La Base):** Contiene los valores por defecto para todos los entornos.
2. **`appsettings.{Entorno}.json` (El Ajuste):** Si el entorno es `Development`, se lee `appsettings.Development.json` y se sobrescriben *únicamente* las claves que se hayan declarado allí.
3. **Variables de Entorno del Sistema Operativo:** Tienen la máxima prioridad.

El SDK Web y los Archivos de Configuración

- En los proyectos de **Consola** estándar, si agregamos un archivo externo como `appsettings.json`, debíamos modificar manualmente el archivo `.csproj` agregando la directiva `<CopyToOutputDirectory>` para que el archivo viajara junto al ejecutable.
- En los proyectos **WEB** cualquier archivo `appsettings.json` o `appsettings.{Entorno}.json` es copiado a la carpeta de salida (`bin/...`) al compilar o ejecutar, sin que tengamos que escribir una sola línea de configuración en el `.csproj`.



Experimento con los Ciclos de Vida de los servicios registrados en el contenedor Web



Nos falta ver el comportamiento del registro **AddScoped**.

Preparar el experimento: en el proyecto **Aplicacion**, crear una interfaz que exponga el **Id** del objeto.

Archivo: IOperacion.cs (en proyecto Aplicacion)

```
namespace Hosting.Aplicacion;
```

```
public interface IOperacion  
{  
    Guid Id { get; }  
}
```

```
// Interfaces "marcadoras" para pedir el ciclo de vida exacto  
public interface IOperacionTransient : IOperacion { }  
public interface IOperacionScoped : IOperacion { }  
public interface IOperacionSingleton : IOperacion { }
```

Copiar el código del archivo
11 Teoria-Recursos



Implementando la operación



Ahora implementamos todas las interfaces en una sola clase. El truco aquí es que el **Guid** se generará **una única vez por cada objeto** que se construya.

Archivo: `Operacion.cs` (en proyecto Aplicacion)

```
namespace Hosting.Aplicacion;

public class Operacion : IOperacionTransient,
    IOperacionScoped, IOperacionSingleton
{
    public Guid Id { get; } = Guid.NewGuid();
}
```

`Guid.NewGuid()` se ejecuta
ÚNICAMENTE cuando se hace un
'new' de esta clase

Copiar el código del archivo
11 Teoria-Recursos



Centralizando la lógica en la capa de Aplicación



Codificar el siguiente Servicio **en proyecto Aplicacion**

```
namespace Hosting.Aplicacion;

// El contenedor inyectará automáticamente estas 6 dependencias
// al instanciar la clase usando este constructor primario.
public class MonitorCicloDeVidaService(
    IOperacionTransient t1, IOperacionTransient t2,
    IOperacionScoped sc1, IOperacionScoped sc2,
    IOperacionSingleton s1, IOperacionSingleton s2)
{
    public object ObtenerTodosLosIds()
    {
        return new { Transient1 = t1.Id, Transient2 = t2.Id,
                     Scoped1 = sc1.Id, Scoped2 = sc2.Id,
                     Singleton1 = s1.Id, Singleton2 = s2.Id };
    }
}
```

Copiar el código del archivo
11 Teoria-Recursos

Retornamos un "tipo anónimo" de C# con los valores recolectados. Cuando este objeto llegue al endpoint de nuestra Web API, ASP.NET Core lo transformará automáticamente a formato JSON.



Registrar los servicios en la program.cs de la Web API



```
using Hosting.Aplicacion;

var builder = WebApplication.CreateBuilder(args);

// Registramos las operaciones
builder.Services.AddTransient<IOperacionTransient, Operacion>();
builder.Services.AddScoped<IOperacionScoped, Operacion>();
builder.Services.AddSingleton<IOperacionSingleton, Operacion>();

// Registramos el servicio monitor (puede ser Transient o Scoped)
builder.Services.AddTransient<MonitorCicloDeVidaService>();

var app = builder.Build();

app.MapGet("/", (MonitorCicloDeVidaService monitor) => monitor.ObtenerTodosLosIds());

app.Run();
```



Registrar los servicios en la program.cs de la Web API



```
using Hosting.Applicacion;
```

```
var builder = Host.CreateApplicationBuilder(args);
```

```
// Registrar los servicios
```

```
builder.Services.AddTransient<MonitorCicloDeVidaService>();
```

```
builder.Services.AddTransient<MonitorCicloDeVidaService>();
```

```
builder.Services.AddTransient<MonitorCicloDeVidaService>();
```

Regla importante: Como el servicio

MonitorCicloDeVidaService

depende de componentes con ciclo de vida **Scoped**, jamás podría ser registrado como **Singleton**.

```
builder.Services.AddTransient<MonitorCicloDeVidaService>();
```

```
builder.Services.AddTransient<MonitorCicloDeVidaService>();
```

```
builder.Services.AddTransient<MonitorCicloDeVidaService>();
```

```
// Registramos el servicio monitor (puede ser Transient o Scoped)
```

```
builder.Services.AddTransient<MonitorCicloDeVidaService>();
```

```
var app = builder.Build();
```

```
app.MapGet("/", (MonitorCicloDeVidaService monitor) => monitor.ObtenerTodosLosIds());
```

```
app.Run();
```



Codificar el endpoint con inyección de parámetros



```
using Hosting.Aplicacion;

var builder = WebApplication.CreateBuilder(args);

// Registramos las operaciones
builder.Services.AddTransient<IOperacionTransient, Operacion>();
builder.Services.AddScoped<IOperacionScoped, Operacion>();
builder.Services.AddSingleton<IOperacionSingleton, Operacion>();

// Registramos el servicio monitor (puede ser Transient o Scoped)
builder.Services.AddTransient<MonitorCicloDeVidaService>();

var app = builder.Build();

app.MapGet("/", (MonitorCicloDeVidaService monitor) => monitor.ObtenerTodosLosIds());

app.Run();
```



Codificar el endpoint con inyección de parámetros



¿Cómo llega el objeto **monitor** ahí?

- El contenedor de dependencias de **ASP.NET Core** no solo trabaja con constructores.
- Cuando definimos una ruta (**endpoint**), el framework analiza los parámetros de la función anónima. Si el tipo de dato (en este caso **MonitorCicloDeVidaService**) está registrado en el contenedor, el framework lo resuelve y nos pasa la instancia de forma automática.

```
using
```

```
var bu
```

```
// Reg
```

```
builder
```

```
builder
```

```
builder
```

```
// Reg
```

```
builder
```

```
var app = builder.Build();
```

```
app.MapGet("/", (MonitorCicloDeVidaService monitor) => monitor.ObtenerTodosLosIds());
```

```
app.Run();
```

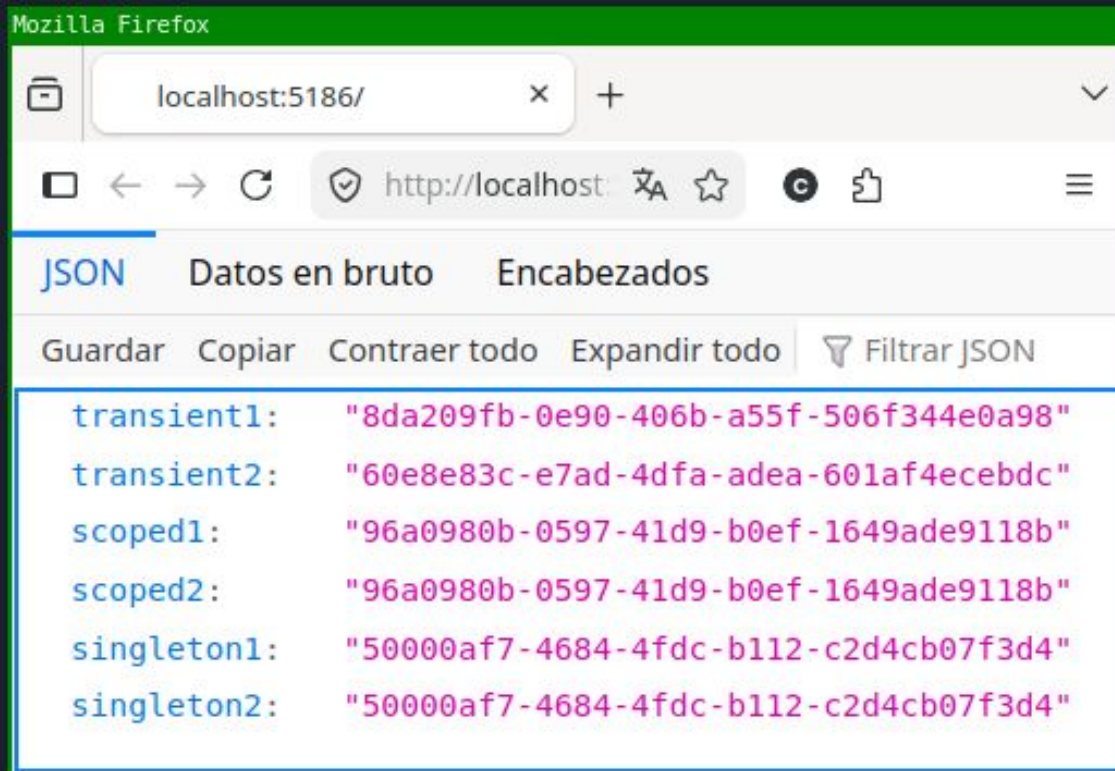




Probando el alcance de los servicios registrados



- Ejecutar el proyecto **ApiHost** (recordar usar el botón de **Play** con el **Program.cs** activo) y abrir el navegador en la URL asignada (ej: <http://localhost:5186/>)





Analicemos el JSON devuelto en esta petición:

- **Transient 1 y 2:** Los IDs son **diferentes**. El contenedor hizo dos `new Operacion()`.
- **Scoped 1 y 2:** Los IDs son **iguales**. Durante esta petición `HTTP`, el contenedor entregó la misma instancia a quien la pidió.
- **Singleton 1 y 2:** Los IDs son **iguales**.



JSON Datos en bruto Encabezados

Guardar Copiar Contraer todo Expandir todo Filtrar JSON

```
transient1: "8da209fb-0e90-406b-a55f-506f344e0a98"
transient2: "60e8e83c-e7ad-4dfa-adea-601af4ecebdc"
scoped1:    "96a0980b-0597-41d9-b0ef-1649ade9118b"
scoped2:    "96a0980b-0597-41d9-b0ef-1649ade9118b"
singleton1: "50000af7-4684-4fdc-b112-c2d4cb07f3d4"
singleton2: "50000af7-4684-4fdc-b112-c2d4cb07f3d4"
```




La prueba de fuego



Presionar **F5** en el navegador para simular a otro usuario entrando, o una **nueva petición HTTP** al servidor. Comparemos con la respuesta anterior:

The diagram shows two screenshots of a Mozilla Firefox browser window displaying JSON data from localhost:5186/. A yellow arrow labeled 'F5' points from the first screenshot to the second, indicating a refresh action.

First Screenshot (Before F5):

```
transient1: "8da209fb-0e90-406b-a55f-506f344e0a98"
transient2: "60e8e83c-e7ad-4dfa-adea-601af4ecebdc"
scoped1: "96a0980b-0597-41d9-b0ef-1649ade9118b"
scoped2: "96a0980b-0597-41d9-b0ef-1649ade9118b"
singleton1: "50000af7-4684-4fdc-b112-c2d4cb07f3d4"
singleton2: "50000af7-4684-4fdc-b112-c2d4cb07f3d4"
```

Second Screenshot (After F5):

```
transient1: "ef981ab9-5626-40cc-b5f7-9254773562de"
transient2: "fc57daa0-e3b7-46d9-a2dd-a544187e6bb5"
scoped1: "4ebd4ca5-86b9-47e8-8be6-9186227fccaa"
scoped2: "4ebd4ca5-86b9-47e8-8be6-9186227fccaa"
singleton1: "50000af7-4684-4fdc-b112-c2d4cb07f3d4"
singleton2: "50000af7-4684-4fdc-b112-c2d4cb07f3d4"
```

- **Transient:** Cambiaron ambos. (Comportamiento esperado).
- **Scoped:** **Cambiaron respecto a la petición anterior.** Al iniciar un nuevo HTTP Request, el framework creó un "Scope" nuevo. ¡Así es como logramos aislar la información y las transacciones entre distintos usuarios conectados al mismo tiempo!
- **Singleton:** **Idénticos.** Jamás cambiarán hasta que reiniciemos la aplicación.



Realizar la siguiente prueba



Aunque sabemos que no debe hacerse, registrar **MonitorCicloDeVidaService** como Singleton ejecutar y analizar qué sucede.

```
using Hosting.Aplicacion;

var builder = WebApplication.CreateBuilder(args);

// Registramos las operaciones
builder.Services.AddTransient<IOperacionTransient, Operacion>();
builder.Services.AddScoped<IOperacionScoped, Operacion>();
builder.Services.AddSingleton<IOperacionSingleton, Operacion>();

// Registramos el servicio monitor (puede ser Transient o Scoped)
builder.Services.AddSingleton<MonitorCicloDeVidaService>();

var app = builder.Build();

app.MapGet("/", (MonitorCicloDeVidaService monitor) => monitor.ObtenerTodosLosIds());

app.Run();
```



Realizar la siguiente prueba



Aunque sabemos que no debe hacerse, registrar **MonitorCicloDeVidaService** como Singleton ejecutar y analizar qué sucede.

```
using Hosting.Aplicacion;

var builder = WebApplication.CreateBuilder(args);

// Registramos las operaciones
builder.Services.AddTransient<IOperacionTransient, Operacion>();
builder.Services.AddScoped<IOperacionScoped, Operacion>();
builder.Services.AddSingleton<IOperacionSingleton, Operacion>();

// Registramos el servicio monitor (puede ser Transient o Scoped)
builder.Services.AddSingleton<MonitorCicloDeVidaService>();

var app = builder.Build();
```

Exception has occurred: CLR/System.AggregateException ×

An unhandled exception of type 'System.AggregateException' occurred in Microsoft.Extensions.DependencyInjection.dll: 'Some services are not constructed'

Inner exceptions found, see \$exception in variables window for more details.

Innermost exception System.InvalidOperationException : Cannot consume scoped service 'Hosting.Aplicacion.IOperacionScoped' from singleton 'Hosting.Aplicacion.MonitorCicloDeVidaService'.

at Microsoft.Extensions.DependencyInjection.ServiceLookup.CallSiteValidator.VisitCallSite(ServiceCallSite callSite, CallSiteValidatorSta

at Microsoft.Extensions.DependencyInjection.ServiceLookup.CallSiteValidator.VisitConstructor(ConstructorCallSite constructorCallSite, C

Dependencias Cautivas y Ciclos de Vida

La única restricción estricta: Un `Singleton` **NO** puede depender de un `Scoped`.

- **Protección:** `ASP.NET Core` aborta el arranque de la aplicación lanzando `InvalidOperationException` en `builder.Build()`.

¿Por qué ocurre esto?

- El `Scoped` aísla los datos de *una sola petición HTTP*.
- Si un `Singleton` lo captura, lo retiene de por vida, y entonces podría compartir los datos de un usuario con todos los demás.

Regla general de diseño:

- Aunque el resto de las combinaciones están permitidas, **un servicio de vida larga no debería depender de uno más corto** para evitar retenerlo en memoria más allá de su propósito original.

Fin teoría 11

Práctica sobre la teoría 11

Ejercicio 1:

1. Crear una solución con dos proyectos: **Aplicación** y **ConsolaHost**. En el proyecto **Aplicación**, crear una clase **ProcesadorArchivos** que implemente una interfaz **IProcesador**. Esta clase debe simular el procesamiento de archivos en un directorio.
2. Su comportamiento debe ser configurable. Crear una clase llamada **ProcesadorOptions** con dos propiedades: **DirectorioEntrada** (string) y **MaxArchivosPorLote** (int). Inyectar esta clase de opciones en el constructor de **ProcesadorArchivos**.
3. En el método **Procesar()** de este servicio, imprimir en consola un mensaje que diga: *"Procesando un máximo de {MaxArchivosPorLote} archivos desde la ruta: {DirectorioEntrada}"*.
4. En el proyecto **ConsolaHost**, agregar las configuraciones correspondientes en el archivo **appsettings.json**. y crear la clase **AppRunner** (similar a como se hizo en esta teoría)
5. Modificar el **Program.cs** para registrar estas opciones, registrar el servicio **IProcesador**, e inyectarlo en la clase **AppRunner**.
6. Ejecutar la aplicación y verificar que el mensaje muestre los valores leídos del JSON.

Ejercicio 2:

1. Crear una solución con dos proyectos: **Aplicación** y **ApiHost**. En el proyecto **Aplicacion**, crear un servicio llamado **ServicioNotificacion** y registrarlo en la API como **Scoped** (simulando que necesita datos específicos de la petición actual del usuario).
2. Crear un servicio llamado **GestorAlertasGlobales** y registrarlo como **Singleton** (simulando un proceso central que vive siempre).
3. Inyectar el **ServicioNotificacion** dentro del constructor de **GestorAlertasGlobales**.
4. Intentar inyectar **GestorAlertasGlobales** en cualquier endpoint de prueba y ejecutar la aplicación.

Pregunta de análisis: ¿Qué excepción arroja la consola al intentar construir el **IHost**? Explicar por qué el framework prohíbe explícitamente esta combinación de ciclos de vida.